Abhinav Kaushik
26MCAP101N055

DATA STRUCTURE LAB

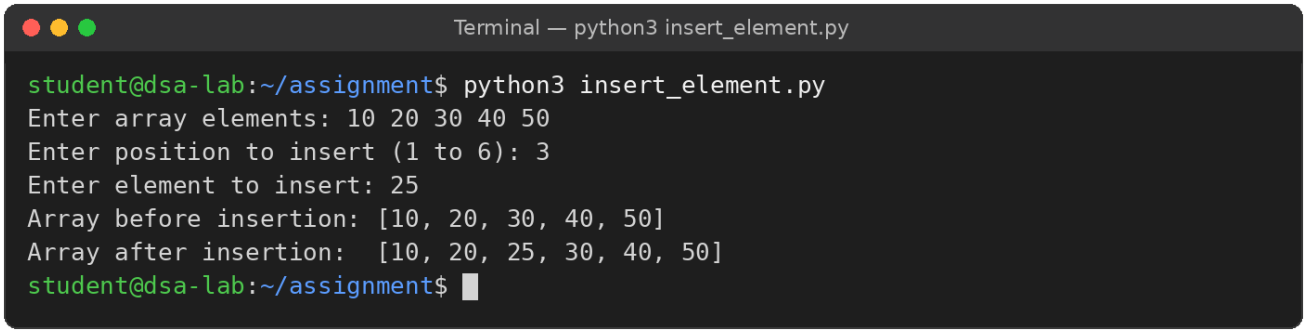
Q1. Insert a new element in an array

Code:

```
arr = list(map(int, input("Enter array elements: ").split()))
pos = int(input("Enter position to insert (1 to {}): ".format(len(arr) + 1)))
val = int(input("Enter element to insert: "))

print("Array before insertion:", arr)
if 1 <= pos <= len(arr) + 1:
    arr.append(0)
    for i in range(len(arr) - 1, pos - 1, -1):
        arr[i] = arr[i - 1]
    arr[pos - 1] = val
    print("Array after insertion: ", arr)
else:
    print("Invalid position")
```

Output:



Q2. Delete an element from an array

Code:

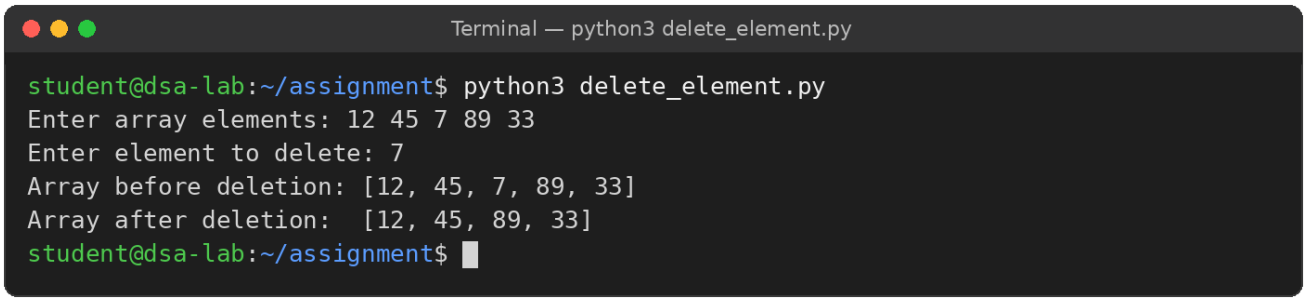
```
arr = list(map(int, input("Enter array elements: ").split()))
val = int(input("Enter element to delete: "))

print("Array before deletion:", arr)
pos = -1
for i in range(len(arr)):
    if arr[i] == val:
        pos = i
        break

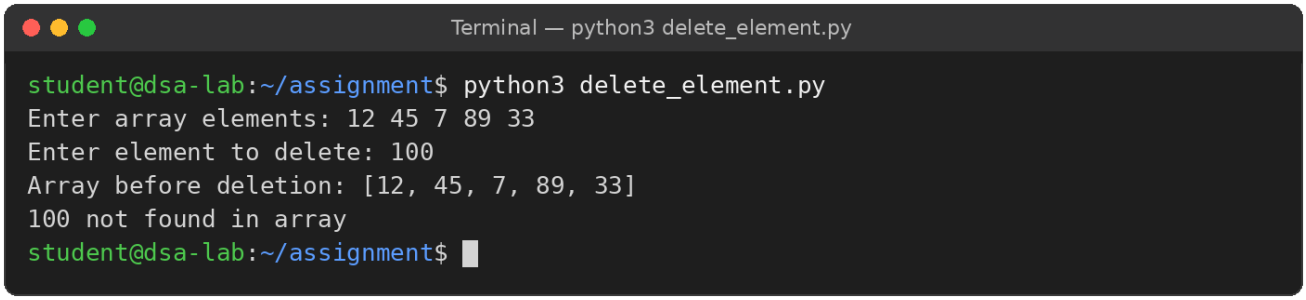
if pos == -1:
    print(val, "not found in array")
else:
    for i in range(pos, len(arr) - 1):
        arr[i] = arr[i + 1]
    arr.pop()
    print("Array after deletion: ", arr)
```

Output:

Run 1



Run 2



Q3. Find the product of two matrices

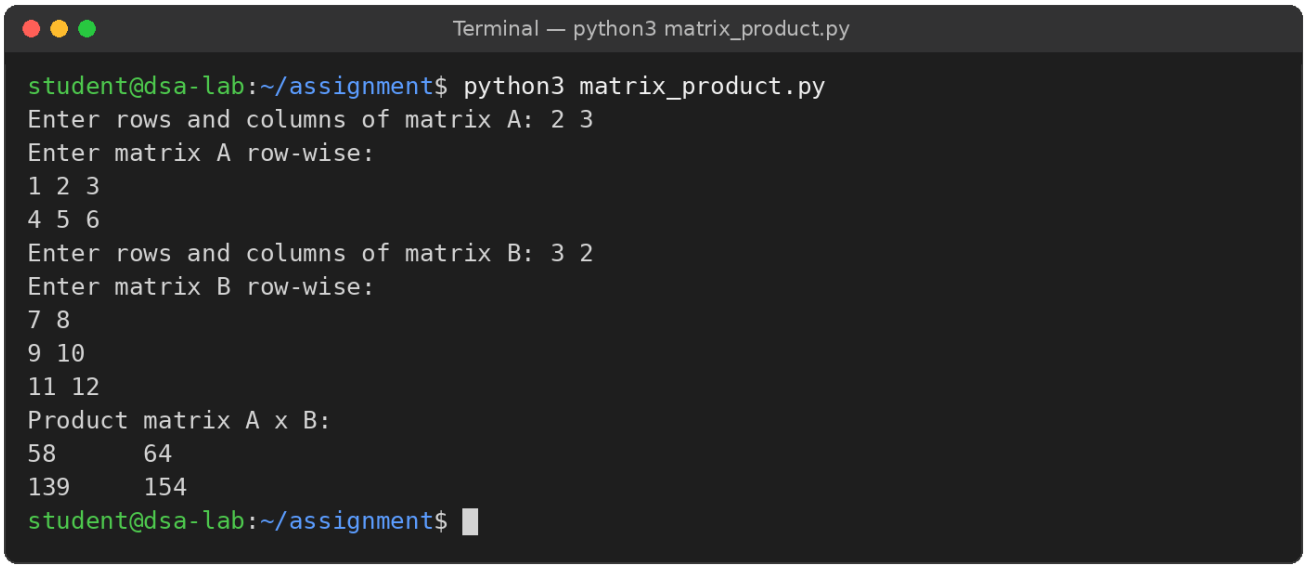
Code:

```
r1, c1 = map(int, input("Enter rows and columns of matrix A: ").split())
print("Enter matrix A row-wise:")
A = [list(map(int, input().split())) for _ in range(r1)]

r2, c2 = map(int, input("Enter rows and columns of matrix B: ").split())
print("Enter matrix B row-wise:")
B = [list(map(int, input().split())) for _ in range(r2)]

if c1 != r2:
    print("Multiplication not possible: columns of A must equal rows of B")
else:
    C = [[0] * c2 for _ in range(r1)]
    for i in range(r1):
        for j in range(c2):
            for k in range(c1):
                C[i][j] += A[i][k] * B[k][j]
    print("Product matrix A x B:")
    for row in C:
        print(*row, sep="\t")
```

Output:



Q4. Sum of diagonal and non-diagonal elements in a matrix

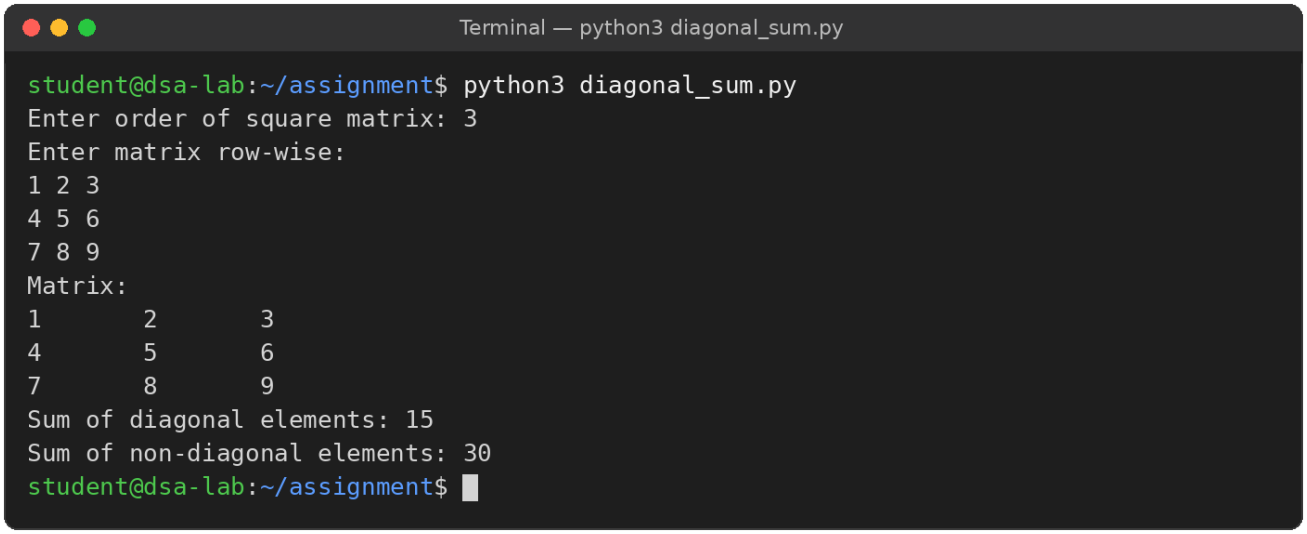
Code:

```
n = int(input("Enter order of square matrix: "))
print("Enter matrix row-wise:")
M = [list(map(int, input().split())) for _ in range(n)]

diag = non_diag = 0
for i in range(n):
    for j in range(n):
        if i == j:
            diag += M[i][j]
        else:
            non_diag += M[i][j]

print("Matrix:")
for row in M:
    print(*row, sep="\t")
print("Sum of diagonal elements:", diag)
print("Sum of non-diagonal elements:", non_diag)
```

Output:



Q5. Enter marks of 5 students in 3 subjects each and find each student's percentage

Code:

```
students = []
for i in range(1, 6):
    name = input("Enter name of student {}: ".format(i))
    marks = list(map(int, input("Enter marks in 3 subjects (out of 100): ").split()))
    students.append((name, marks))

print()
head = "{:<10}{:>6}{:>6}{:>6}{:>7}".format("Name", "Sub1", "Sub2", "Sub3", "Total")
print(head + "{:>12}".format("Percentage"))
for name, marks in students:
    total = sum(marks)
    percent = total / 300 * 100
    row = "{:<10}{:>6}{:>6}{:>6}{:>7}".format(name, *marks, total)
    print(row + "{:>11.2f}%".format(percent))
```

Output:

```
Terminal — python3 student_percentage.py

student@dsa-lab:~/assignment$ python3 student_percentage.py
Enter name of student 1: Riya
Enter marks in 3 subjects (out of 100): 85 90 78
Enter name of student 2: Arjun
Enter marks in 3 subjects (out of 100): 72 65 80
Enter name of student 3: Sneha
Enter marks in 3 subjects (out of 100): 95 88 92
Enter name of student 4: Karan
Enter marks in 3 subjects (out of 100): 60 70 55
Enter name of student 5: Priya
Enter marks in 3 subjects (out of 100): 78 82 91

Name        Sub1  Sub2  Sub3  Total  Percentage
Riya          85    90    78    253      84.33%
Arjun         72    65    80    217      72.33%
Sneha         95    88    92    275      91.67%
Karan         60    70    55    185      61.67%
Priya         78    82    91    251      83.67%
student@dsa-lab:~/assignment$
```

Q6.1 Stack - Operations (Push, Pop, Peek, Display)

Code:

```
MAX = 5
stack = []

print("1.Push  2.Pop  3.Peek  4.Display  5.Exit")
while True:
    ch = int(input("\nEnter choice: "))
    if ch == 1:
        if len(stack) == MAX:
            print("Stack Overflow")
        else:
            x = int(input("Enter element: "))
            stack.append(x)
            print(x, "pushed")
    elif ch == 2:
        if not stack:
            print("Stack Underflow")
        else:
            print(stack.pop(), "popped")
    elif ch == 3:
        if stack:
            print("Top element:", stack[-1])
        else:
            print("Stack is empty")
    elif ch == 4:
        if stack:
            print("Stack (top -> bottom):", *stack[::-1])
        else:
            print("Stack is empty")
    elif ch == 5:
        print("Exiting")
        break
    else:
        print("Invalid choice")
```

Output:

```
Terminal — python3 stack_operations.py

student@dsa-lab:~/assignment$ python3 stack_operations.py
1.Push  2.Pop  3.Peek  4.Display  5.Exit

Enter choice: 2
Stack Underflow

Enter choice: 1
Enter element: 10
10 pushed

Enter choice: 1
Enter element: 20
20 pushed

Enter choice: 1
Enter element: 30
30 pushed

Enter choice: 4
Stack (top -> bottom): 30 20 10

Enter choice: 3
Top element: 30

Enter choice: 2
30 popped

Enter choice: 4
Stack (top -> bottom): 20 10

Enter choice: 1
Enter element: 40
40 pushed

Enter choice: 1
Enter element: 50
50 pushed
```

```
Terminal — python3 stack_operations.py

Enter choice: 1
Enter element: 60
60 pushed

Enter choice: 1
Stack Overflow

Enter choice: 4
Stack (top -> bottom): 60 50 40 20 10

Enter choice: 5
Exiting
student@dsa-lab:~/assignment$
```

Q6.2 Stack - Reverse of a string

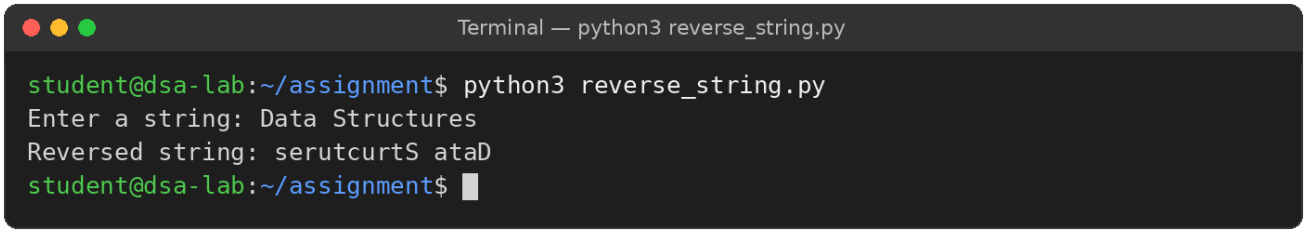
Code:

```
s = input("Enter a string: ")
stack = []
for ch in s:
    stack.append(ch)

rev = ""
while stack:
    rev += stack.pop()

print("Reversed string:", rev)
```

Output:



Q6.3 Stack - Parenthesis matching

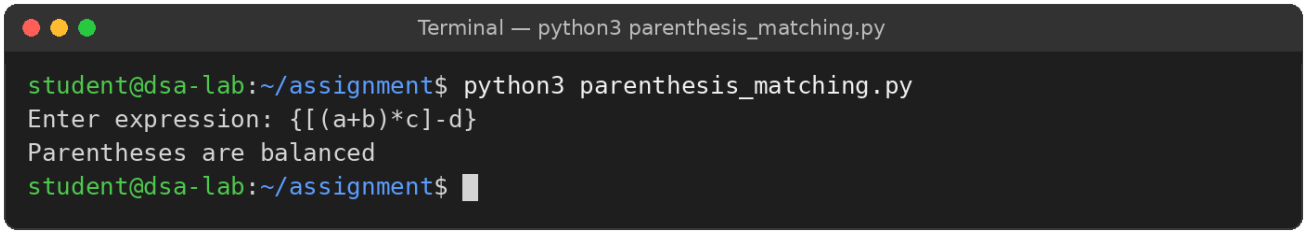
Code:

```
def is_balanced(expr):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

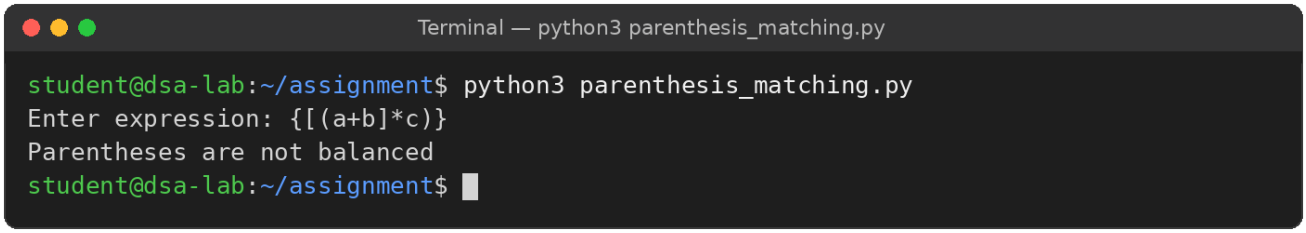
expr = input("Enter expression: ")
if is_balanced(expr):
    print("Parentheses are balanced")
else:
    print("Parentheses are not balanced")
```

Output:

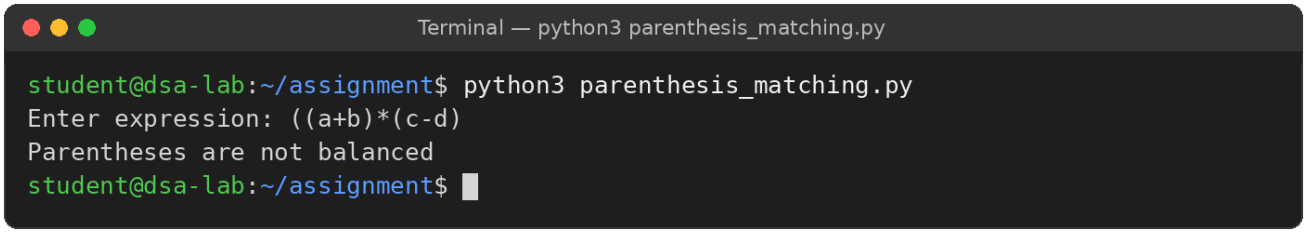
Run 1



Run 2



Run 3



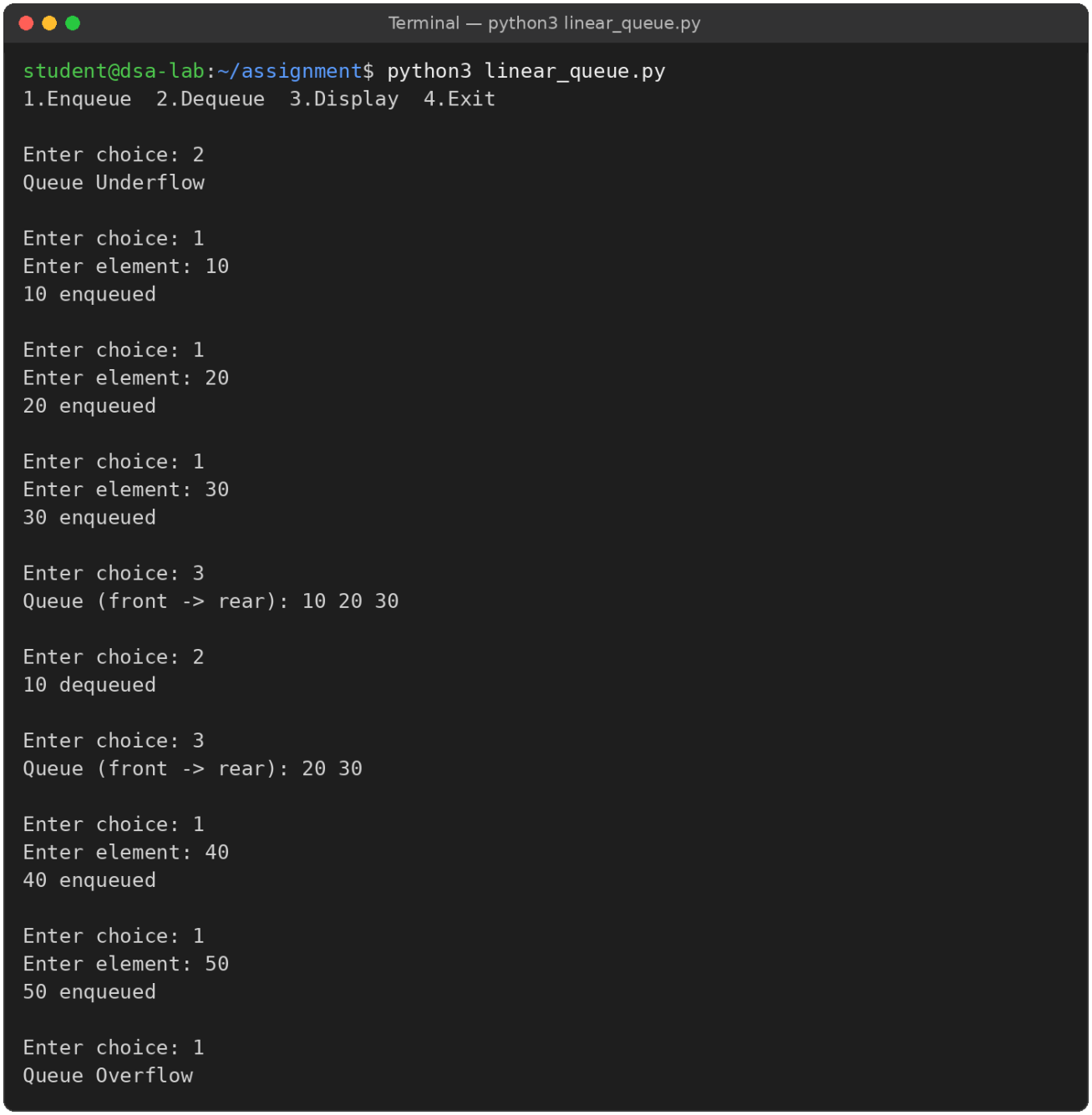
Q7.1 Queue - General (linear) queue operations

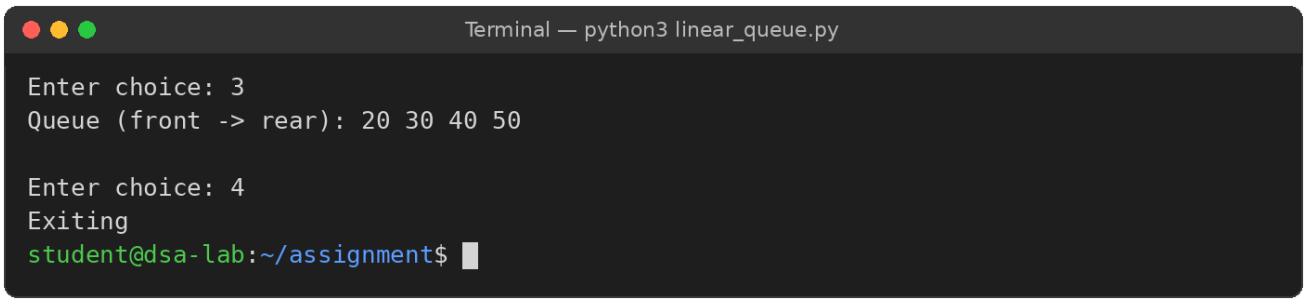
Code:

```
SIZE = 5
q = [None] * SIZE
front = rear = -1

print("1.Enqueue  2.Dequeue  3.Display  4.Exit")
while True:
    ch = int(input("\nEnter choice: "))
    if ch == 1:
        if rear == SIZE - 1:
            print("Queue Overflow")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear += 1
            q[rear] = x
            print(x, "enqueued")
    elif ch == 2:
        if front == -1:
            print("Queue Underflow")
        else:
            print(q[front], "dequeued")
            front += 1
            if front > rear:
                front = rear = -1
    elif ch == 3:
        if front == -1:
            print("Queue is empty")
        else:
            print("Queue (front -> rear):", *q[front:rear + 1])
    elif ch == 4:
        print("Exiting")
        break
    else:
        print("Invalid choice")
```

Output:


Terminal — python3 linear_queue.py
student@dsa-lab:~/assignment$ python3 linear_queue.py
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 2
Queue Underflow
Enter choice: 1
Enter element: 10
10 enqueued
Enter choice: 1
Enter element: 20
20 enqueued
Enter choice: 1
Enter element: 30
30 enqueued
Enter choice: 3
Queue (front -> rear): 10 20 30
Enter choice: 2
10 dequeued
Enter choice: 3
Queue (front -> rear): 20 30
Enter choice: 1
Enter element: 40
40 enqueued
Enter choice: 1
Enter element: 50
50 enqueued
Enter choice: 1
Queue Overflow


Terminal — python3 linear_queue.py
Enter choice: 3
Queue (front -> rear): 20 30 40 50
Enter choice: 4
Exiting
student@dsa-lab:~/assignment$

Q7.2 Queue - Circular queue operations

Code:

```
SIZE = 5
q = [None] * SIZE
front = rear = -1

print("1.Enqueue  2.Dequeue  3.Display  4.Exit")
while True:
    ch = int(input("\nEnter choice: "))
    if ch == 1:
        if (rear + 1) % SIZE == front:
            print("Circular Queue is Full")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear = (rear + 1) % SIZE
            q[rear] = x
            print(x, "enqueued at index", rear)
    elif ch == 2:
        if front == -1:
            print("Circular Queue is Empty")
        else:
            print(q[front], "dequeued from index", front)
            if front == rear:
                front = rear = -1
            else:
                front = (front + 1) % SIZE
    elif ch == 3:
        if front == -1:
            print("Circular Queue is Empty")
        else:
            items = []
            i = front
            while True:
                items.append(q[i])
                if i == rear:
                    break
                i = (i + 1) % SIZE
            print("Queue (front -> rear):", *items)
            print("front =", front, " rear =", rear)
    elif ch == 4:
        print("Exiting")
        break
    else:
        print("Invalid choice")
```

Output:

Terminal — python3 circular_queue.py

```
student@dsa-lab:~/assignment$ python3 circular_queue.py
1.Enqueue  2.Dequeue  3.Display  4.Exit

Enter choice: 1
Enter element: 10
10 enqueued at index 0

Enter choice: 1
Enter element: 20
20 enqueued at index 1

Enter choice: 1
Enter element: 30
30 enqueued at index 2

Enter choice: 1
Enter element: 40
40 enqueued at index 3

Enter choice: 1
Enter element: 50
50 enqueued at index 4

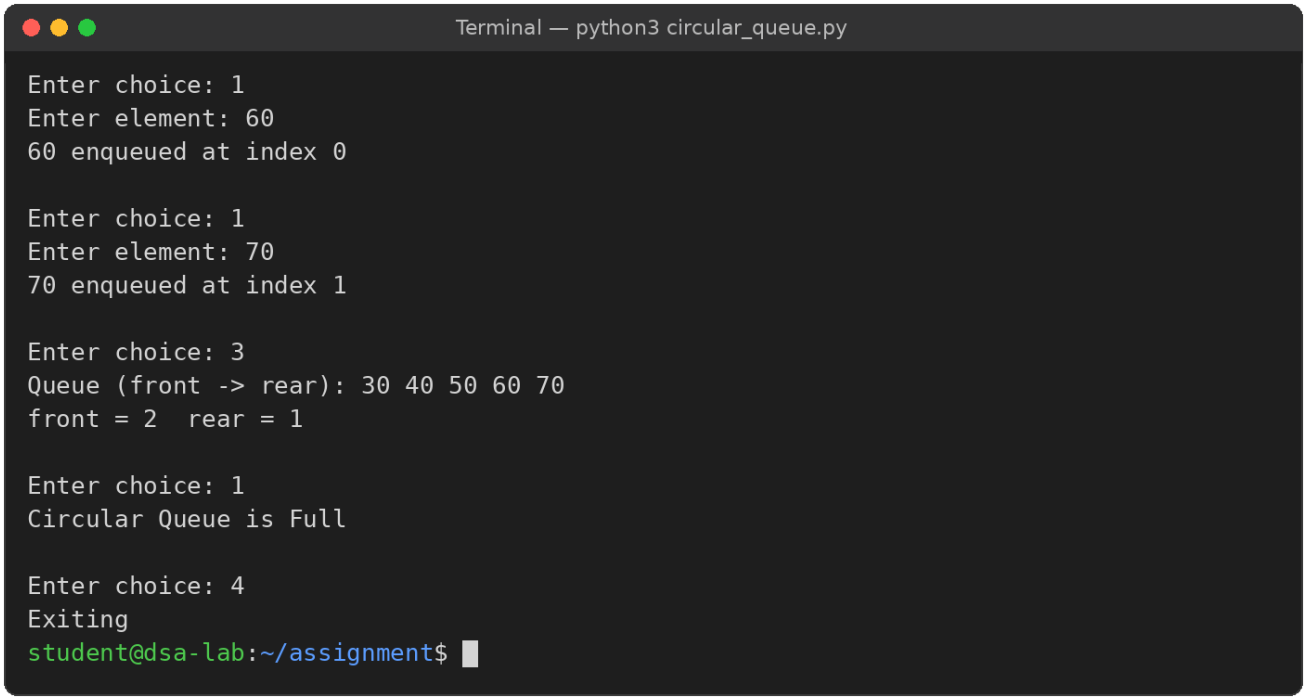
Enter choice: 1
Circular Queue is Full

Enter choice: 3
Queue (front -> rear): 10 20 30 40 50
front = 0  rear = 4

Enter choice: 2
10 dequeued from index 0

Enter choice: 2
20 dequeued from index 1

Enter choice: 3
Queue (front -> rear): 30 40 50
front = 2  rear = 4
```


Terminal — python3 circular_queue.py
Enter choice: 1
Enter element: 60
60 enqueued at index 0
Enter choice: 1
Enter element: 70
70 enqueued at index 1
Enter choice: 3
Queue (front -> rear): 30 40 50 60 70
front = 2 rear = 1
Enter choice: 1
Circular Queue is Full
Enter choice: 4
Exiting
student@dsa-lab:~/assignment$